

Polishing <chrono>

Difference from previous revision

Previous revision: [P0092R0](#)

- Per request of the LEWG, removed the increment/decrement operator for `time_point`.
- Replaced "largest" with "greatest" in `floor Returns:`.
- Replaced "smallest" with "least" in `ceil Returns:`.
- Use "equidistant" in `round Returns:`.
- Use "`%2 == 0`" for "even" in `round Returns:`.
- Use `numeric_limits<duration>::is_signed` in `abs Remarks:`.
- Replace "instantiation" with "specialization" in all `Remarks:`.

Contents

- [Introduction](#)
- [Implementation](#)
- [Wording](#)

Introduction

This paper proposes a few minor additions to the chrono library to make it easier to use and more uniform:

- Alternative rounding modes for durations and time_points:
 - `floor`: round towards negative infinity.
 - `ceil`: round towards positive infinity.
 - `round`: round towards nearest, to even on tie.
- `abs` *only* for signed duration types.

These are *minor* additions that make chrono easier to use. They have been implemented **and** used for years. They should have been part of the original chrono proposal for C++11, but simply slipped through the cracks by no one's fault but my own.

The rounding modes differ from `duration_cast` and `time_point_cast` in that these existing rounding functions truncate towards zero: downwards for positive values and upwards for negative values.

Implementation

The implementation is shown below for the utility functions. This presentation is meant to:

- Fully inform the intent of this proposal.
- Immediately convey the scope and the small-ness of this proposal.
- Show the ease of implementation of this proposal.

This implementation is consistent with that which has been up on [my personal website](#) for years.

```

namespace std { namespace chrono {

namespace detail
{

template <class T>
struct is_duration
    : public false_type

```

```

    {};
```

$$\text{template <class Rep, class Period>}$$

$$\text{struct is_duration<duration<Rep, Period>>}$$

$$\text{: public true_type}$$

$$\text{\{};$$

$$\text{\}} \text{ // namespace detail}$$

$$\text{template <class To, class Rep, class Period,}$$

$$\text{class = enable_if_t<detail::is_duration<To>\{\}\>>}$$

$$\text{constexpr}$$

$$\text{To}$$

$$\text{floor(const duration<Rep, Period>\& d)}$$

$$\text{\{}$$

$$\text{To t = duration_cast<To>(d);}$$

$$\text{if (t > d)}$$

$$\text{--t;}$$

$$\text{return t;}$$

$$\text{\}}$$

$$\text{template <class To, class Rep, class Period,}$$

$$\text{class = enable_if_t<detail::is_duration<To>\{\}\>>}$$

$$\text{constexpr}$$

$$\text{To}$$

$$\text{ceil(const duration<Rep, Period>\& d)}$$

$$\text{\{}$$

$$\text{To t = duration_cast<To>(d);}$$

$$\text{if (t < d)}$$

$$\text{++t;}$$

$$\text{return t;}$$

$$\text{\}}$$

$$\text{template <class To, class Rep, class Period,}$$

$$\text{class = enable_if_t<detail::is_duration<To>\{\}$$

$$\text{\&\& !treat_as_floating_point<typename To::rep>\{\}\>>}$$

$$\text{constexpr}$$

$$\text{To}$$

$$\text{round(const duration<Rep, Period>\& d)}$$

$$\text{\{}$$

$$\text{To t0 = floor<To>(d);}$$

$$\text{To t1 = t0 + To\{1\};}$$

$$\text{auto diff0 = d - t0;}$$

$$\text{auto diff1 = t1 - d;}$$

$$\text{if (diff0 == diff1)}$$

$$\text{\{}$$

$$\text{if (t0.count() \& 1)}$$

$$\text{return t1;}$$

$$\text{return t0;}$$

$$\text{\}}$$

$$\text{else if (diff0 < diff1)}$$

$$\text{return t0;}$$

$$\text{return t1;}$$

$$\text{\}}$$

$$\text{template <class To, class Clock, class FromDuration,}$$

$$\text{class = enable_if_t<detail::is_duration<To>\{\}\>>}$$

$$\text{constexpr}$$

$$\text{time_point<Clock, To>}$$

$$\text{floor(const time_point<Clock, FromDuration>\& tp)}$$

$$\text{\{}$$

$$\text{return time_point<Clock, To>\{floor<To>(tp.time_since_epoch())\};}$$

$$\text{\}}$$

$$\text{template <class To, class Clock, class FromDuration,}$$

$$\text{class = enable_if_t<detail::is_duration<To>\{\}\>>}$$

$$\text{constexpr}$$

$$\text{time_point<Clock, To>}$$

$$\text{ceil(const time_point<Clock, FromDuration>\& tp)}$$

$$\text{\{}$$

$$\text{return time_point<Clock, To>\{ceil<To>(tp.time_since_epoch())\};}$$

$$\text{\}}$$

$$\text{template <class To, class Clock, class FromDuration,}$$

$$\text{class = enable_if_t<detail::is_duration<To>\{\}$$

$$\text{\&\& !treat_as_floating_point<typename To::rep>\{\}\>>}$$

$$\text{constexpr}$$

$$\text{time_point<Clock, To>}$$

$$\text{round(const time_point<Clock, FromDuration>\& tp)}$$

```

{
    return time_point<Clock, To>{round<To>(tp.time_since_epoch())};
}

template <class Rep, class Period,
          class = enable_if_t
            <
                duration<Rep, Period>::min() < duration<Rep, Period>::zero()
            > >
constexpr
duration<Rep, Period>
abs(duration<Rep, Period> d)
{
    return d >= d.zero() ? d : -d;
}

} } // namespace std::chrono

```

Wording

- Add to 20.12.2 Header <chrono> synopsis [time.syn], under duration_cast:

```

template <class ToDuration, class Rep, class Period>
constexpr ToDuration floor(const duration<Rep, Period>& d);

template <class ToDuration, class Rep, class Period>
constexpr ToDuration ceil(const duration<Rep, Period>& d);

template <class ToDuration, class Rep, class Period>
constexpr ToDuration round(const duration<Rep, Period>& d);

```

- Add to 20.12.2 Header <chrono> synopsis [time.syn], under time_point_cast:

```

template <class ToDuration, class Clock, class Duration>
constexpr time_point<Clock, ToDuration>
floor(const time_point<Clock, Duration>& tp);

template <class ToDuration, class Clock, class Duration>
constexpr time_point<Clock, ToDuration>
ceil(const time_point<Clock, Duration>& tp);

template <class ToDuration, class Clock, class Duration>
constexpr time_point<Clock, ToDuration>
round(const time_point<Clock, Duration>& tp);

```

- Add a new section to 20.12.2 Header <chrono> synopsis [time.syn] (*specialized algorithms*: — within namespace std::chrono):

```

template <class Rep, class Period>
constexpr duration<Rep, Period> abs(duration<Rep, Period> d)

```

- Add to 20.12.5.7 <duration_cast> [time.duration.cast]:

```

template <class ToDuration, class Rep, class Period>
constexpr ToDuration floor(const duration<Rep, Period>& d);

```

Remarks: This function shall not participate in overload resolution unless ToDuration is a specialization of duration.

Returns: The greatest result t representable in ToDuration for which $t \leq d$.

```

template <class ToDuration, class Rep, class Period>
constexpr ToDuration ceil(const duration<Rep, Period>& d);

```

Remarks: This function shall not participate in overload resolution unless ToDuration is a specialization of duration.

Returns: The least result t representable in ToDuration for which $t \geq d$.

```

template <class ToDuration, class Rep, class Period>
constexpr ToDuration round(const duration<Rep, Period>& d);

```

Remarks: This function shall not participate in overload resolution unless ToDuration is a specialization of duration, and treat_as_floating_point<typename ToDuration::rep>::value is false.

Returns: The value of ToDuration that is closest to d . If there are two closest values, then return the

value t for which $t \% 2 == 0$.

- Add a new section to 20.12.5 [time.duration]: duration algorithms [time.duration.alg].

```
template <class Rep, class Period>
constexpr duration<Rep, Period> abs(duration<Rep, Period> d)
```

Remarks: This function shall not participate in overload resolution unless `numeric_limits<Rep>::is_signed` is true.

Returns: If $d \geq d.zero()$, return d , otherwise return $-d$.

- Add to 20.12.6.7 time_point_cast [time.point.cast]:

```
template <class ToDuration, class Clock, class Duration>
constexpr time_point<Clock, ToDuration>
floor(const time_point<Clock, Duration>& tp);
```

Remarks: This function shall not participate in overload resolution unless `ToDuration` is a specialization of `duration`.

Returns: `time_point<Clock, ToDuration>{floor<ToDuration>(tp.time_since_epoch())}`.

```
template <class ToDuration, class Clock, class Duration>
constexpr time_point<Clock, ToDuration>
ceil(const time_point<Clock, Duration>& tp);
```

Remarks: This function shall not participate in overload resolution unless `ToDuration` is a specialization of `duration`.

Returns: `time_point<Clock, ToDuration>{ceil<ToDuration>(tp.time_since_epoch())}`.

```
template <class ToDuration, class Clock, class Duration>
constexpr time_point<Clock, ToDuration>
round(const time_point<Clock, Duration>& tp);
```

Remarks: This function shall not participate in overload resolution unless `ToDuration` is a specialization of `duration`, and `treat_as_floating_point<typename ToDuration::rep>::value` is false.

Returns: `time_point<Clock, ToDuration>{round<ToDuration>(tp.time_since_epoch())}`.

- Modify 20.12.5.5 duration non-member arithmetic [time.duration.nonmember]:

```
template <class Rep1, class Period, class Rep2>
constexpr duration<common_type_t<Rep1, Rep2>, Period>
operator/(const duration<Rep1, Period>& d, const Rep2& s);
```

Remarks: This operator shall not participate in overload resolution unless `Rep2` is implicitly convertible to `CR(Rep1, Rep2)` and `Rep2` is not ~~an instantiation~~ a specialization of `duration`.

...

```
template <class Rep1, class Period, class Rep2>
constexpr duration<common_type_t<Rep1, Rep2>, Period>
operator%(const duration<Rep1, Period>& d, const Rep2& s);
```

Remarks: This operator shall not participate in overload resolution unless `Rep2` is implicitly convertible to `CR(Rep1, Rep2)` and `Rep2` is not ~~an instantiation~~ a specialization of `duration`.

- Modify 20.12.5.7 duration_cast [time.duration.cast]:

```
template <class ToDuration, class Rep, class Period>
constexpr ToDuration duration_cast(const duration<Rep, Period>& d);
```

Remarks: This operator shall not participate in overload resolution unless `ToDuration` is ~~an instantiation~~ a specialization of `duration`.

- Modify 20.12.6.7 time_point_cast [time.point.cast]:

```
template <class ToDuration, class Clock, class Duration>
constexpr time_point<Clock, ToDuration>
time_point_cast(const time_point<Clock, Duration>& t);
```

Remarks: This operator shall not participate in overload resolution unless `ToDuration` is ~~an~~

~~instantiation~~ a specialization of duration.